

AWS API Gateway Lab Part 4: Test the Full Workflow with Postman

To Begin with the Lab

Summary of the Lab

In this final part of the lab, the complete **API Gateway** workflow was tested end to end using **Postman**. The **DynamoDB** backend, three **AWS Lambda** functions, and the **HTTP API** routes were already prepared in the previous parts, so this step focused on validating the full request flow from an external client. A POST request was used to create a user record, a GET request was used to retrieve the record by `userId`, and a DELETE request was used to remove the record from **UsersTable**. Each request was sent through the API Gateway invoke URL, and the responses were verified in both Postman and DynamoDB. After testing, the lab noted that the **HTTP API** could be deleted if needed, while keeping the Lambda functions and DynamoDB table for future use.

Understanding the Final API Test

This final step exists to prove that the whole application flow works outside AWS, just like a real client app would use it. **Postman** acts as the test client and sends HTTP requests to **API Gateway**, which then forwards them to the correct **Lambda** function. This confirms that the API routes, Lambda logic, and DynamoDB operations are connected correctly. It also shows why **API Gateway** is needed: the frontend should not call Lambda directly, and the gateway becomes the secure entry point for the application.

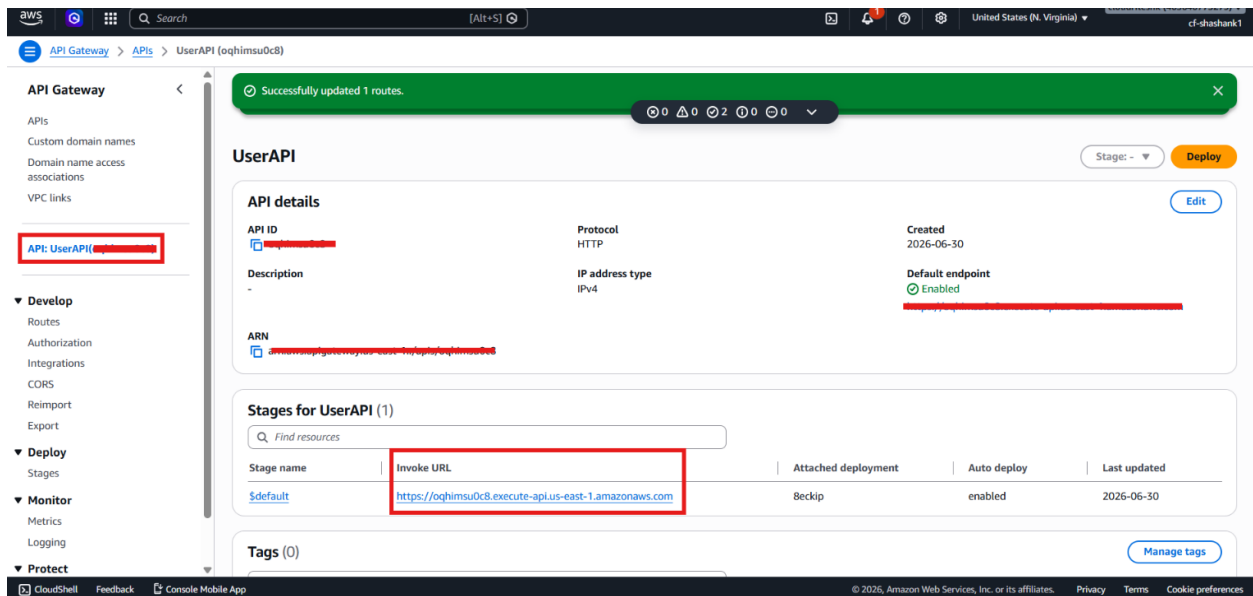
Example:

A mobile app wants to add a new user named Kabir. It sends a POST request to `/user`, API Gateway invokes `createUserFunction`, and the Lambda function stores the record in **UsersTable**. Later, the app can call `GET /user/101` to read the record and `DELETE /user/101` to remove it.

Lab Steps

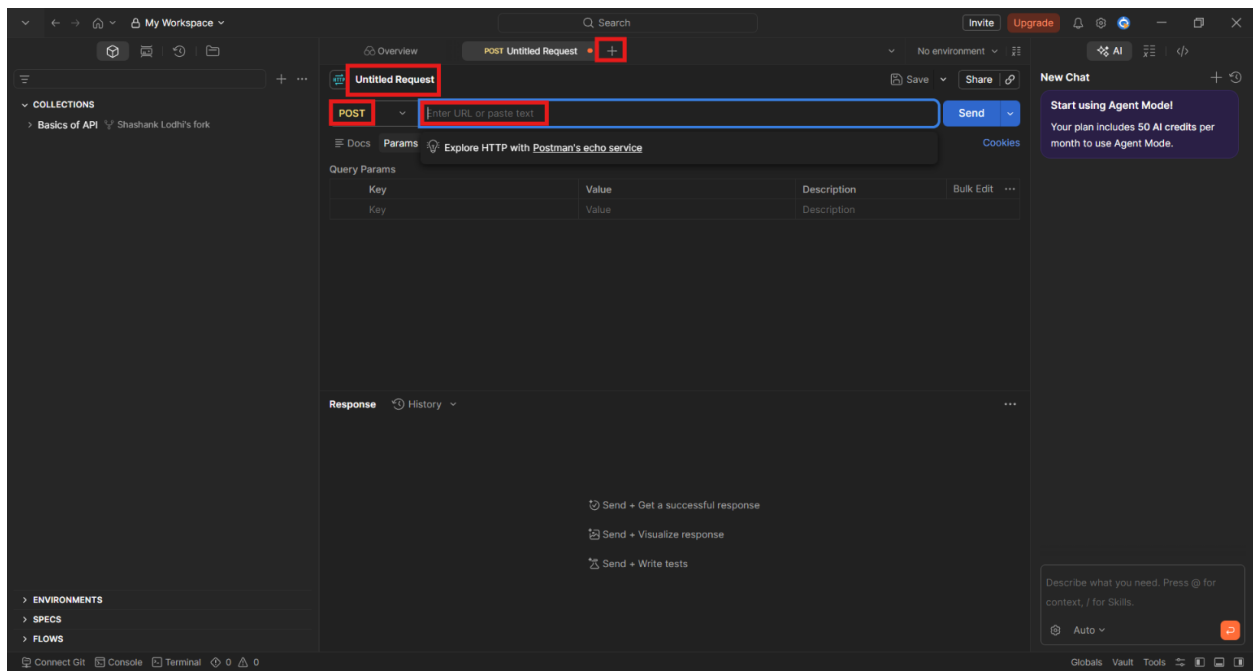
Step 1 – Open the API Gateway Invoke URL

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Select **UserAPI**.
- Go to **Stages**.
- Copy the **Invoke URL**.

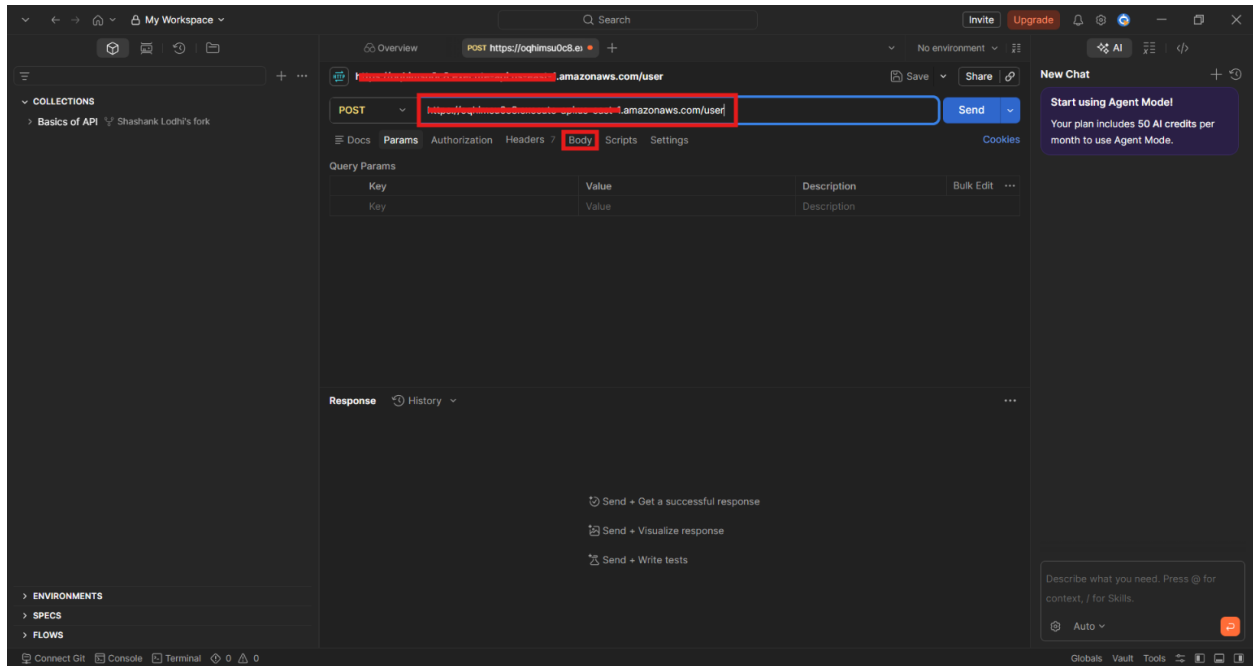


Step 2 – Test the Create User Request in Postman

- Open Postman.
- Create a new request.
- Set the method to POST.



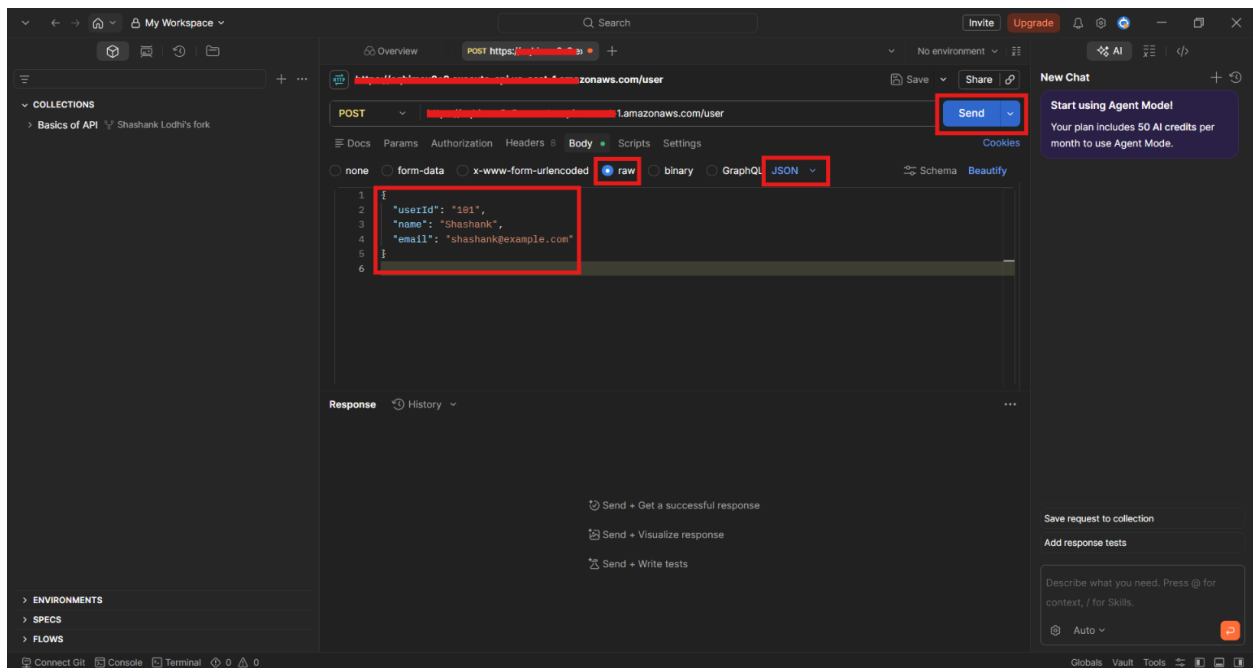
- Paste the invoke URL and append /user.
- Open the **Body** tab.



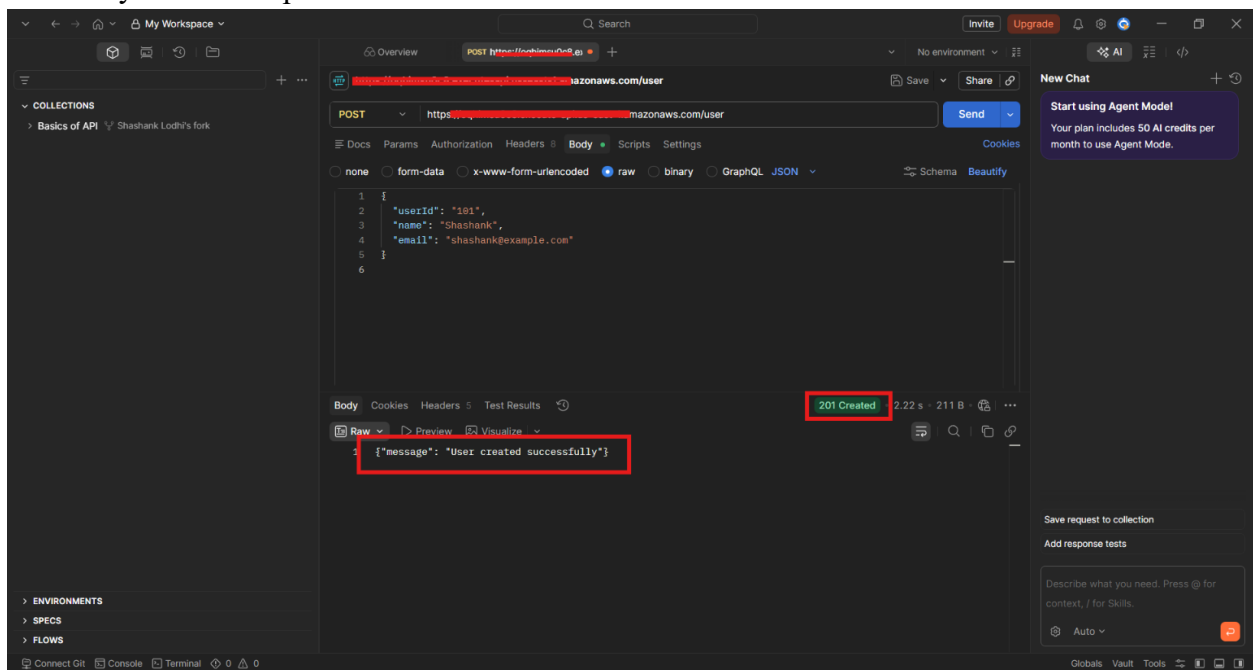
- Select **raw**.
- Choose **JSON** as the format.
- Add the user payload.

```
{  
  "userId": "101",  
  "name": "Shashank",  
  "email": "shashank@example.com"  
}
```

- Click **Send**.



- Verify that the response shows user creation success.



- Open Amazon DynamoDB.
- Go to **Tables** → **UsersTable**.

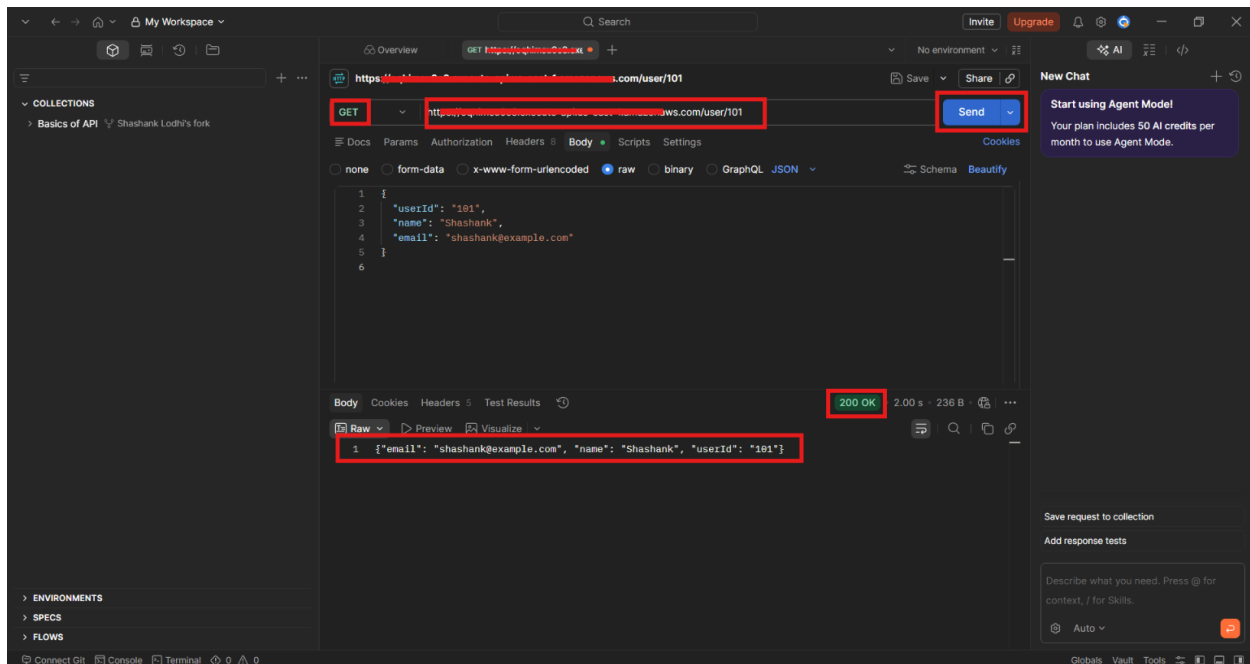
The screenshot shows the AWS Management Console for a DynamoDB instance. On the left, the 'DynamoDB' sidebar is visible. The main area displays the 'UsersTable' details. A red box highlights the 'Explore table items' button in the top right corner of the table's header. Below this, the 'General information' section shows the table's configuration, including the partition key 'userid (String)', capacity mode 'On-demand', and table status 'Active'. The 'Read/write capacity' section is also visible at the bottom.

- Click **Explore table items**.
- Confirm that the record was inserted.

The screenshot shows the 'Explore items' view for the 'UsersTable'. The 'Scan' operation is selected, and the results are displayed in a table. A green banner at the top indicates the scan is 'Completed' with 1 item returned. The scan results table has columns for 'userid (String)', 'email', and 'name'. The first row shows '101', 'shashank@...', and 'Shashank'. A red box highlights this row. The 'Actions' and 'Create item' buttons are visible in the top right of the results section.

Step 3 – Test the Get User Request

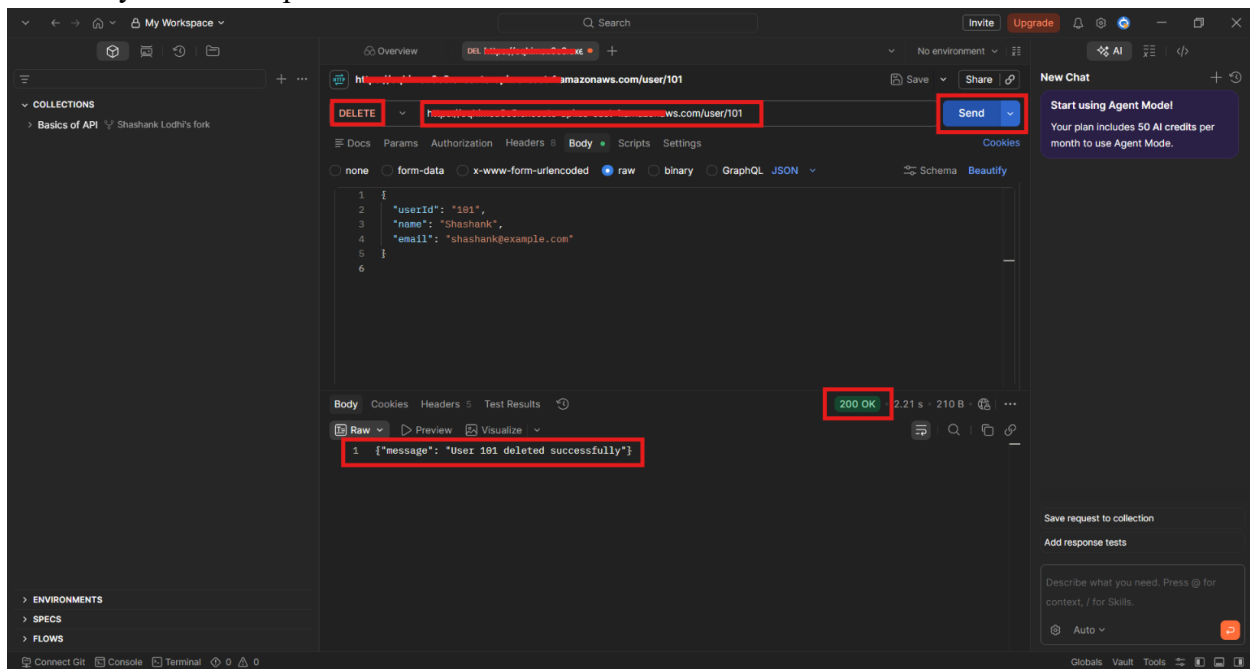
- In **Postman**, change the method to **GET**.
- Use the route `/user/101`.
- Click **Send**.
- Verify that the user details are returned.



- Confirm that the data matches the record stored in **UsersTable**.

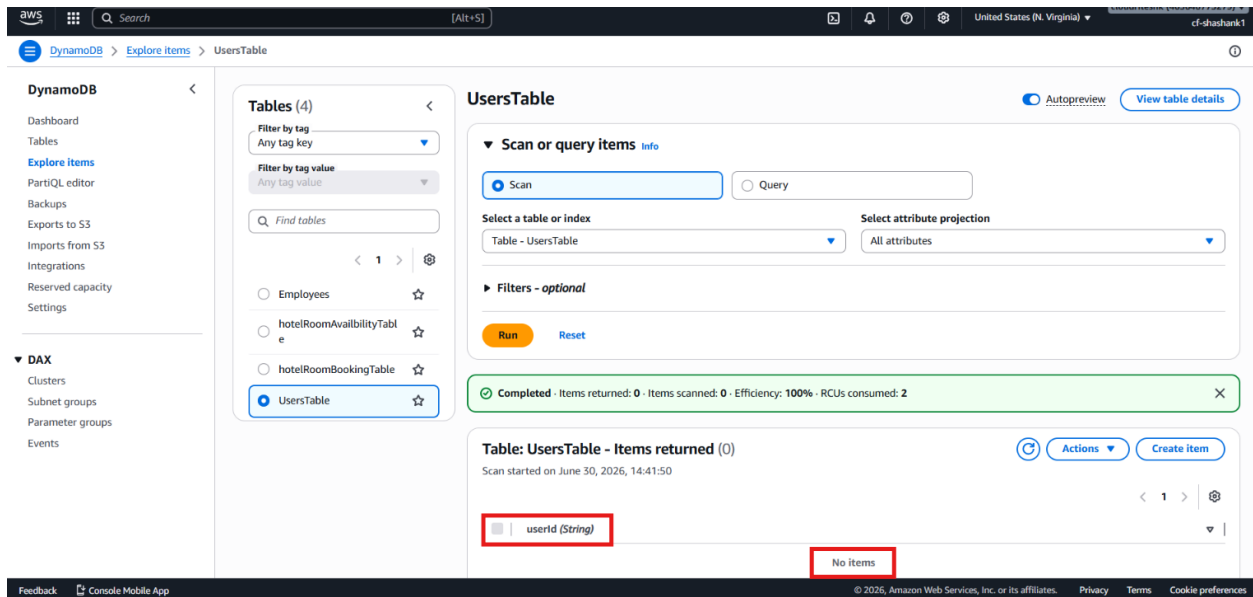
Step 4 – Test the Delete User Request

- In **Postman**, change the method to **DELETE**.
- Use the route `/user/101`.
- Click **Send**.
- Verify that the response confirms deletion.



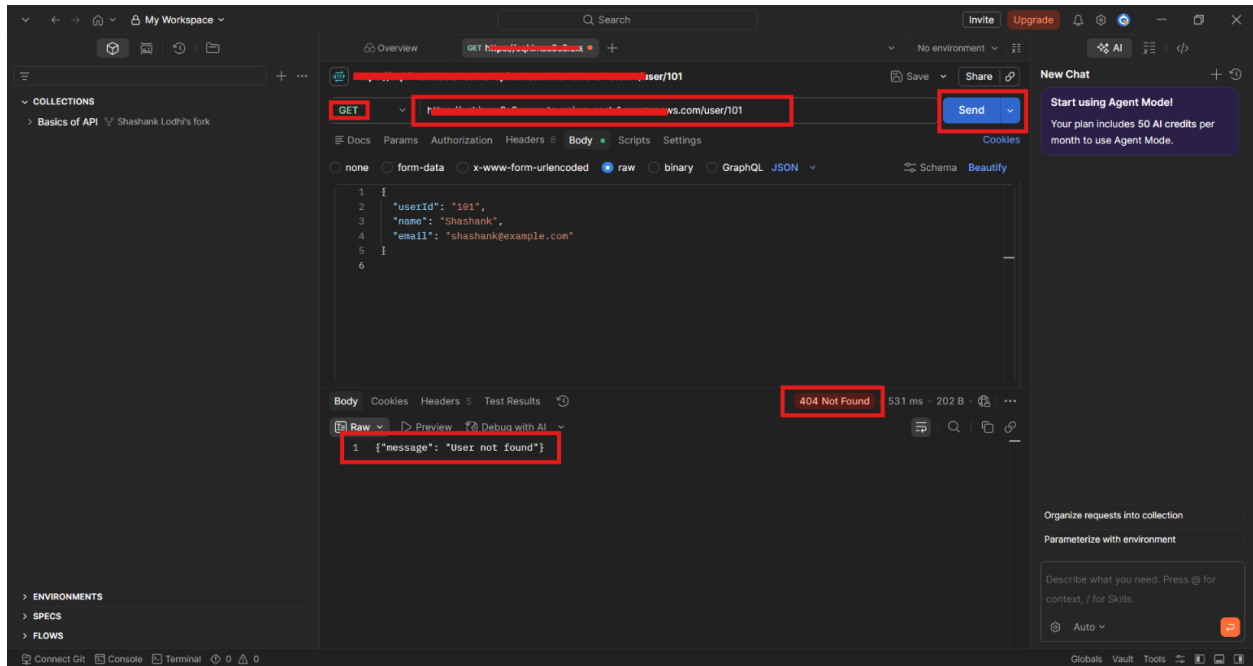
- Return to **Amazon DynamoDB**.
- Refresh **UsersTable**.

- Confirm that the record was removed.



Step 5 – Confirm the Error Case

- Send the same GET /user/101 request again after deletion.
- Verify that the response shows **User not found**.



- Confirm that the API correctly handles missing records.

Step 6 – Clean Up the API Gateway Resource

- Return to API Gateway.
- Select the **HTTP API** if you want to remove it.
- Delete the API Gateway resource only if the lab is complete.

- Keep the **Lambda** functions and **DynamoDB** table if you want to reuse them later.

Verification

- The **HTTP API** was tested successfully from **Postman**.
- The POST route created a record in **UsersTable**.
- The GET route returned the correct user details.
- The DELETE route removed the record successfully.
- The missing-record case returned **User not found**.
- The complete request flow from client → **API Gateway** → **Lambda** → **DynamoDB** was verified.